

# RAG System for Apple's Q3 2022 Form 10-Q

## *Approach and Methodology*

### 1. What I built, and why

The assignment was to build a retrieval-augmented generation system that answers questions about Apple's third-quarter fiscal 2022 Form 10-Q, a 28-page filing. The hard part isn't wiring an LLM to a vector store — it's that a 10-Q isn't really “text.” A meaningful part of it is tables (the income statement, segment breakdowns, the share-repurchase schedule), there's a smaller amount of narrative discussion and legal text, and there's one embedded image. A financial answer that gets the right number but the wrong period, or the wrong year, or invents a figure that isn't actually there, is worse than useless. So the whole design is built around three things: retrieve the right evidence, don't let the model guess past it, and always show where an answer came from.

At a high level, the pipeline parses the PDF into typed elements (text, tables, images), chunks them, embeds them into Chroma, and answers questions with a hybrid dense+BM25 retriever that feeds a strict, citation-only prompt to a Groq-hosted model:

PDF -> parse (text / tables / images) -> chunk -> embed -> ChromaDB

Question -> hybrid retrieval (dense + BM25, RRF) -> top-k chunks

-> grounded prompt -> Groq LLM -> cited answer -> Streamlit UI

| Stage                     | Technology                                 |
|---------------------------|--------------------------------------------|
| Text and table extraction | pdfplumber                                 |
| Image extraction          | PyMuPDF                                    |
| Embeddings                | BAAI/bge-small-en-v1.5 (local, 384-dim)    |
| Vector store              | ChromaDB, persistent, cosine similarity    |
| Keyword retrieval         | BM25 (rank-bm25)                           |
| Rank fusion               | Reciprocal Rank Fusion (RRF)               |
| Answer generation         | openai/gpt-oss-120b on Groq, temperature 0 |
| Interface                 | Streamlit                                  |

On the 25-question evaluation set, the correct page shows up in the top 6 retrieved chunks for 23 of the 24 answerable questions, every citation the model produces actually points at something it retrieved, and it correctly refuses the one unanswerable question.

### 2. Pipeline

Ingestion (scripts/ingest.py) runs once; answering runs per question through the Streamlit app.

**Parsing.** parser.py turns the PDF into a flat list of elements, each tagged with a type, a page number, and a section label — 646 text elements, 31 tables, and 1 image in total. pdfplumber finds the table boxes on each page; whatever text falls outside one of those boxes gets grouped into lines and merged with the tables in reading order, so every element knows the heading above it (PART I, Item 2., Note 9 –, matched against standard 10-Q conventions with regex). That heading is what ends up in the citation later.

Tables get converted to Markdown, with two clean-ups that mattered more than I expected going in. First, pdfplumber sometimes splits a currency symbol from its number into adjacent cells (“\$”, “63,355”), so those get re-merged, along with dropping empty rows and columns. Second, and this was the bigger issue: the column headers that actually

distinguish “three months” from “nine months” and “2022” from “2021” often sit just above the table, outside the box pdfplumber detects. Without them, a table is just a grid of numbers with no way to know what they mean. I fixed this by grabbing the three text lines immediately before each table and attaching them as a [Context: ...] prefix.

Images are pulled out with PyMuPDF and saved along with their page, size, and format. There's exactly one in this filing — the Apple logo — so this path isn't heavily exercised here, but it's in place for filings that use more figures.

**Chunking.** chunker.py turns the elements into 164 chunks — 132 text, 31 table, 1 image — and treats each type differently. Tables become one chunk apiece and are never split, so a number never gets separated from the row and column labels that give it meaning. Text is grouped by section and by page (page matters here too: a chunk that spanned two pages could end up citing the wrong one), then split into roughly 800-character pieces with 100 characters of overlap, cutting at sentence boundaries where possible so a fact near a chunk boundary doesn't get orphaned. Every chunk opens with a [Source: <section>, Page N] header, which is what lets the model cite its source directly instead of me reconstructing it after the fact.

**Embeddings and storage.** Chunks are embedded with BAAI/bge-small-en-v1.5 (384-dim, normalized) and written to a persistent ChromaDB collection using cosine distance, with kind, page, and section stored as metadata. Indexing uses upsert, so re-running ingestion after a code change doesn't duplicate the collection. Queries get the “Represent this sentence for searching relevant passages: ...” instruction prefix that BGE recommends; passages don't.

**Hybrid retrieval.** This is the part I spent the most time tuning. Dense retrieval alone is good at matching meaning but weak on exact tokens — a filing is full of things like “Greater China” and “14,604” that embeddings can under-rank relative to something that merely sounds similar. So I added a BM25 keyword index over the same chunks and combined the two with Reciprocal Rank Fusion: each ranker returns its top 48 candidates, and a chunk's fused score is  $1/(60 + \text{rank\_dense} + 1) + 1/(60 + \text{rank\_bm25} + 1)$ . RRF works on ranks rather than raw scores, so I didn't have to normalize cosine similarity against a BM25 score, which would otherwise have been its own headache.

One thing hybrid retrieval on its own didn't fix: for a question like “what were net sales,” prose that discusses net sales can outrank the table that actually contains the number, because the prose often repeats the query's exact phrasing more densely than a table does. To correct for this, questions that look like a financial lookup (a financial term plus a phrase like “what was” or “how much”) give table chunks a flat 0.015 bonus, plus another 0.010–0.015 if the query and the table share a specific metric phrase such as “net sales.” Fused scores are small — a chunk ranked first by both rankers scores around 0.033 — so this boost is a fairly strong nudge toward tables, but it only applies to lookup-style financial questions, so it doesn't distort something like the Epic Games question.

The top k chunks (6 by default, adjustable from 2–10 in the UI) go to the generator.

**Answer generation.** Retrieved chunks are numbered and labeled with their type, page, and section, then sent to openai/gpt-oss-120b on Groq at temperature 0. The system prompt does most of the work against hallucination: answer only from the retrieved context, check every retrieved table for the right row and column, keep period (three vs. nine months), year (2022 vs. 2021), and unit (dollars, percent, shares) straight, report every value a comparison question asks for instead of collapsing periods into one invented combined number, avoid doing math the context doesn't directly support, say “The information is not available in the retrieved context” when it genuinely isn't there, and answer in one short paragraph ending with exactly one citation in the form (Source: Page X, Section Name).

**Interface.** The Streamlit app takes a typed or selected question, shows the answer, and — this turned out to matter more during development than I expected — has an expandable panel listing every retrieved chunk with its page, section, and fusion score. Table chunks render as actual tables through one generic Markdown-to-dataframe function rather than per-table code, with the raw extracted text shown alongside the rendered version so extraction errors are easy to spot. Being able to see retrieval and generation separately is what let me tell, for a wrong answer, whether retrieval had missed the page or the model had misread a table it was actually given.

### 3. Design decisions worth explaining

Treating text, tables, and images as distinct element types, instead of flattening everything to plain text, is the one decision the rest of the system depends on — a table flattened into prose loses exactly the row/column structure financial questions need, so nothing downstream (chunking, the table boost, the Markdown renderer) would work without it.

I picked RRF over a weighted score combination or a learned reranker mainly for simplicity: RRF needs no score normalization and has essentially one parameter, whereas a cross-encoder reranker is a model I'd have to add, run, and justify for a 164-chunk corpus. It's a reasonable next step (Section 7), just not a first-pass one.

For the LLM, I actually tried a small model running locally through Ollama before settling on Groq. The local model was fine on plain narrative questions but noticeably worse at reading tables correctly and sticking to the citation format under the strict prompt, so I dropped the local backend and kept Groq as the only path. That's the one place in this project where I compared two options empirically rather than choosing on cost and setup grounds — everything else (the embedding model, pdfplumber, Chroma) was picked because it's free, runs on a laptop with no GPU, and is easy for a grader to reproduce, not because I benchmarked it against paid alternatives like OpenAI embeddings or a hosted vector database. Those would be worth revisiting if a larger evaluation set showed retrieval misses actually caused by embedding quality, which so far it doesn't.

The generic Markdown-to-dataframe table renderer in the UI replaced an earlier version with hard-coded layouts for two of the sample tables — that worked for the demo questions but would have broken on anything else, so I rebuilt it as one function that handles any table pdfplumber can extract.

### 4. Evaluation

evaluation/questions.json has 25 questions — 24 answerable, 1 deliberately not — built to probe specific failure modes rather than sampled at random. Nine are headline-financial lookups (net sales, R&D, operating income, EPS, and so on), five hit the segment/geography table in Note 9, three cover margins and tax, two are narrative MD&A questions, two concern the Epic Games litigation, two are about share repurchases, and one tests whether the system gets the accounting basis right. The set deliberately includes traps: three-month vs. nine-month columns sitting side by side, 2022 vs. 2021 figures in adjacent columns, a repurchase question where the filing reports three sub-period average prices and no single overall average, and a handful of paraphrase pairs — two different phrasings of the same net-sales question, for instance — to check that retrieval isn't just keying off wording.

evaluate.py scores retrieval alone (does one of the top-6 retrieved chunks come from an expected page), and evaluate\_generation.py runs the full pipeline and scores the generated answer against a reference.

| Metric          | Result                                                             |
|-----------------|--------------------------------------------------------------------|
| Retrieval Hit@1 | 16/24 (0.667)                                                      |
| Retrieval Hit@3 | 21/24 (0.875)                                                      |
| Retrieval Hit@6 | 23/24 (0.958)                                                      |
| Retrieval MRR   | 0.771                                                              |
| Numerical match | 0.798 (0.964 after correcting mis-transcribed references, see 4.4) |
| Citation match  | 24/24 (1.000)                                                      |
| Abstention      | 1/1 (1.000)                                                        |

I also ran seven representative questions by hand through the UI (net sales, Greater China, R&D, Epic Games, repurchases, gross margin, Services revenue) and checked the cited answers against the filing myself; all seven were correct.

The numerical-match number is a blunt instrument and undersells the system slightly. When I went back and checked the reference answers against the PDF directly, four of them (Q13–Q16, the nine-month segment figures for Americas, Europe, Japan, and Rest of Asia Pacific) turned out to be mis-transcribed in my own answer key — the system's answers were correct and the reference wasn't. Fixing those references and re-scoring the same saved answers gives 0.964, with 23 of 24 fully matching. The one genuine shortfall is Q05: the system correctly reports 142,674 thousand shares but doesn't repeat the three sub-period average prices the reference lists, because the filing doesn't actually give a single average price and the model says so rather than inventing one — arguably the right behavior even though it costs points on the automatic metric.

The one retrieval “miss,” Q09 (operating income), is really a labeling artifact: only page 4 is marked as the expected page, but the generator found the figure on a different retrieved page and answered correctly anyway. Hit@1 sitting well below Hit@6 (0.667 vs. 0.958) tells me the right page is usually retrieved but not always ranked first — two questions only reach it at rank 6 — which is the clearest evidence in the whole evaluation for adding a reranker. Groq's free tier also rate-limited the evaluation run once; the script retries with backoff, so that's a one-off API hiccup rather than a pipeline bug.

I'd treat this evaluation as a sanity check rather than a real benchmark: it's a small set, some questions are paraphrases of each other, and I used the same questions during development and for reporting, all of which push the numbers in an optimistic direction. The “hit” metric is also page-level rather than chunk-level, so it measures whether retrieval found the right neighborhood, not whether the exact passage came back, and there's exactly one abstention question, so that metric really only shows the system passed this one case.

## 5. Assumptions

The system assumes a digitally generated, text-based PDF (no OCR path), a standard SEC 10-Q structure that the section-detection regex can key off (PART, Item, Note headings), and a single-document scope — the collection name, chunk IDs, and prompt are specific to this filing rather than general-purpose. It also assumes users ask factual, English-language questions the filing actually answers, rather than ones needing outside knowledge or multi-step financial modeling; that page-level citation is precise enough traceability for this purpose; that sending filing text to a hosted LLM is acceptable because the filing is public; and that a CPU-only machine with internet access is available, since nothing here needs a GPU.

## 6. Challenges

Most of the real problems turned out to be in extraction, not retrieval or generation. The table-header issue — period labels sitting outside the detected table box — was the first thing that made early answers wrong in a subtle way: the model would read the right numbers but attribute them to the wrong period, because nothing told it which column was which. The [Context: ...] prefix fixed that. Split currency cells and stray empty rows or columns from pdfplumber's table detection were a smaller, more mechanical fix. Citations pointing at the wrong page turned out to be a chunking bug rather than an extraction one — text chunks were originally grouped by section only, so a section spanning two pages could produce a chunk citing the wrong one; grouping by page as well fixed it.

On the retrieval side, the main failure mode was numeric questions pulling back prose that discussed a metric instead of the table that actually held the number, which is what motivated hybrid retrieval and the table boost described above. On the generation side, the model would occasionally blend the three-month and nine-month columns into one figure, or answer confidently about a number that was never actually reported (the share-repurchase “average price”

trap). The strict prompt, temperature 0, and the explicit abstention instruction are what bring that under control, and several evaluation questions were written specifically to test for it.

A couple of smaller engineering issues along the way: the UI initially had hard-coded rendering logic for two of the demo tables, which I replaced with the generic renderer once it was clear that approach wouldn't scale; and after dropping the local-model backend, the evaluation script still had a stale `backend="groq"` argument left over from when there were two backends to choose between, which I caught and fixed while double-checking the evaluation matched the final design. Groq's rate limit hit once during evaluation and was handled by `retry-with-backoff`. And, as noted above, part of “fixing” the evaluation was discovering that some of my own reference answers were wrong and correcting them against the source PDF.

## 7. Limitations and what I'd do next

The biggest limitation is that figures aren't actually understood — the one image in this filing gets extracted and shown in the UI, but its pixels never reach the language model, so there's no real visual question answering here. That's also the most obvious next step: either route extracted images through a vision-capable model, or generate captions for them at ingestion time and index those captions like any other text chunk.

Beyond that, `Hit@1` sitting well below `Hit@6` suggests a cross-encoder reranker over the top ~20 fused candidates would help ranking without much added complexity, and the table boost is a fairly blunt, keyword-driven heuristic tuned on a small set — a learned classifier, or at least a proper held-out tuning pass, would make it more defensible. The system also can't do arithmetic across tables, such as computing a segment growth rate, which would need a small structured layer that loads tables into actual dataframes instead of leaving all reasoning to the LLM. Everything is scoped to this one filing — section patterns, the boost's keyword list, and parts of the prompt would all need generalizing before this could handle a second 10-Q, let alone a different company. And on the evaluation side, a larger question set with a held-out split, every relevant page labeled instead of just one, and some chunk-level or human-judged scoring would give a much more honest picture than the current 25-question set does.

## 8. Reproducing this

Install `requirements.txt`, copy `.env.example` to `.env` and add a Groq API key, then run `python scripts/ingest.py` once to build the index and `streamlit run app/streamlit_app.py` to ask questions. `python -m pytest tests/ -v` runs the 11 offline parsing/chunking tests; `python evaluation/evaluate.py` and `python evaluation/evaluate_generation.py` reproduce the retrieval and generation numbers above. The full repository layout and setup instructions are in `README.md`.

## 9. Conclusion

The system meets its core goal: it preserves table structure, combines semantic and keyword retrieval, and grounds every answer in cited evidence rather than model knowledge. It retrieves the correct page in the top six for 23 of 24 answerable questions and correctly abstains on the unanswerable one. The main limitations are image understanding, the small evaluation set, and ranking quality; each has a clear path for improvement through vision support, a larger held-out evaluation, and reranking. The modular pipeline makes these extensions straightforward.